



RZESZOW UNIVERSITY
OF TECHNOLOGY



THE FACULTY OF
**ELECTRICAL
AND COMPUTER ENGINEERING**
RZESZOW UNIVERSITY OF TECHNOLOGY

Sztuczna Inteligencja

Projekt

Probabilistyczna sieć neuronowa

Spis treści

1. Opis problemu	3
2. Część teoretyczna	3
2.1. Model probabilistycznej sieci neuronowej	3
2.2. Funkcje jądra	5
2.3. Parametr wygładzania σ	6
2.4. Różnice między PNN a innymi sieciami neuronowymi	7
3. Analiza danych	8
4. Skrypt programu	9
5. Eksperymenty	12
5.1. Badanie wpływu funkcji jądra oraz parametru wygładzania σ	12
5.2. Badanie wpływu stałej normalizującej w funkcjach jądra	14
6. Podsumowanie i wnioski	15
Bibliografia	16
Dodatki	16

1. Opis problemu

Zadaniem projektu jest implementacja i analiza probabilistycznej sieci neuronowej (PNN) [1] w kontekście klasyfikacji wiadomości e-mail jako spam lub nie-spam na podstawie zbioru danych spambase [2]. Zostaną również przeprowadzone eksperymenty badające wpływ różnych funkcji jądra i wartości parametru wygładzania σ na dokładność klasyfikacji.

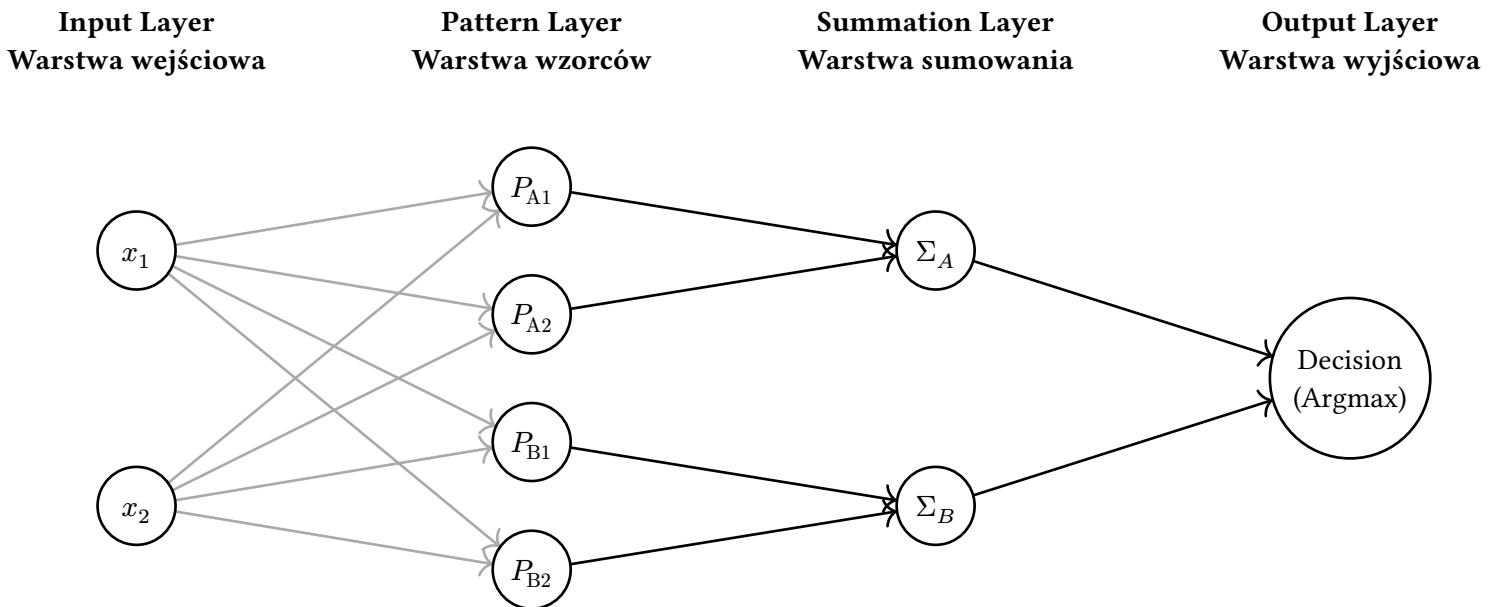
Spam stanowi poważny problem w komunikacji internetowej, powodując obciążenia dla serwerów poczty i negatywne doświadczenie użytkowników. Klasyfikacja wiadomości e-mail jako spam lub nie-spam wymaga rozróżnienia na podstawie charakterystyk takich jak zawartość tekstu, formatowanie i metadane.

Probabilistyczne sieci neuronowe są szczególnie przydatne dla tego problemu ze względu na: brak iteracyjnego uczenia (szybkie trenowanie na zbiorze uczącym), bezpośrednią interpretację decyzji, odporność na szum i zmienność spamu, możliwość wyboru różnych funkcji jądra, oraz asymptotyczną zbieżność do optymalnego klasyfikatora bayesowskiego. Zbiór danych spambase zawiera 4601 wiadomości (około 39,4% spamu) – wystarczającą wielkość dla efektywnego trenowania PNN.

2. Część teoretyczna

Probabilistyczna sieć neuronowa (PNN) [1] jest siecią neuronową, w której wyjście każdego neuronu interpretowane jest jako prawdopodobieństwo przynależności do danej klasy. PNN opiera się na teorii estymacji gęstości jądrowej (ang. **Kernel Density Estimation**, KDE) i jest szeroko stosowana w zadaniach klasyfikacji nadzorowanej.

2.1. Model probabilistycznej sieci neuronowej



Rysunek 1: Model probabilistycznej sieci neuronowej (PNN)

Dla przejrzystości pokazano tylko 2 wzorce treningowe dla każdej z 2 klas oraz $p = 2$ cechy wejściowe

Poszczególne symbole użyte w rysunku 1 oznaczają:

- x_1, x_2 – wejścia sieci (cechy danych wejściowych),
- P_{A1}, P_{A2} – neurony wzorcowe dla klasy A,
- P_{B1}, P_{B2} – neurony wzorcowe dla klasy B,
- Σ_A, Σ_B – neurony sumujące odpowiednio dla klas A i B,
- Decision (Argmax) – neuron decyzyjny przypisujący obserwację do klasy o najwyższej wartości S_k .

2.1.1. Warstwa wejściowa (Input Layer)

Warstwa wejściowa nie wykonuje żadnych operacji obliczeniowych. Jej jedynym zadaniem jest wprowadzenie wektora cech $\mathbf{x} = [x_1, x_2, \dots, x_p]^T \in \mathbb{R}^p$ do sieci oraz dystrybucja jego składowych do wszystkich neuronów warstwy wzorców. Liczba neuronów w tej warstwie jest równa wymiarowości p przestrzeni wejściowej $\mathbf{x}$.

2.1.2. Warstwa wzorców (Pattern Layer)

Warstwa wzorców stanowi bezpośrednią reprezentację zbioru uczącego – każdy neuron przechowuje dokładnie jeden wzorec treningowy $\mathbf{x}_{kj}$, gdzie k oznacza indeks klasy, a j – numer wzorca w obrębie tej klasy.

Dla danego wektora wejściowego $\mathbf{x}$ neuron oblicza odległość euklidesową między wejściem a zapamiętanym wzorcem, a następnie przetwarza ją przez funkcję jądra [1]. Domyślnie stosuje się jądro Gaussa (zob. Rozdział 2.2):

$$\varphi_{kj}(\mathbf{x}) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}_{kj}\|^2}{2\sigma^2}\right) \quad (1)$$

gdzie:

- $\|\mathbf{x} - \mathbf{x}_{kj}\|$ – odległość euklidesowa w przestrzeni cech (zob. Równanie 2),
- σ – parametr wygładzania (**smoothing parameter** lub **bandwidth**), decydujący o szerokości funkcji dzwonnej (zob. Rozdział 2.3).

Odległość euklidesową między wektorem $\mathbf{x} = [x_1, x_2, \dots, x_p]^T$ a wzorcem $\mathbf{x}_{kj}$ można wyrazić wzorem:

$$\|\mathbf{x} - \mathbf{x}_{kj}\| = \sqrt{\sum_{i=1}^p (x_i - x_{kji})^2} \quad (2)$$

gdzie:

- p – liczba cech (wymiarowość przestrzeni wejściowej),
- x_i – i -ta cecha wektora wejściowego,
- x_{kji} – i -ta cecha wzorca $\mathbf{x}_{kj}$.

2.1.3. Warstwa sumowania (Summation Layer)

Każdy neuron sumujący agreguje wyjścia neuronów wzorcowych należących do tej samej klasy [3]:

$$S_k(\mathbf{x}) = \sum_{j=1}^{N_k} \varphi_{kj}(\mathbf{x}) \quad (3)$$

gdzie:

- $S_k(\mathbf{x})$ – wynik działania neuronu sumującego dla klasy k ,
- N_k – liczba wzorców treningowych należących do klasy k ,
- $\varphi_{kj}(\mathbf{x})$ – wyjście neuronu wzorcowego dla wzorca j w klasie k . (Funkcja jądra obliczona w warstwie wzorców).

2.1.4. Warstwa wyjściowa (Output Layer)

Warstwa wyjściowa wybiera klasę o najwyższym wyniku sumującym S_k [4]. Przy równych prawdopodobieństwach **a priori**¹ oraz jednakowych kosztach błędnej klasyfikacji odpowiada to regule **argmax** na wynikach warstwy sumowania (bez dzielenia przez N_k , więc przy nierównych liczebnościach klas nie jest to ścisła estymacja gęstości, lecz skuteczna heurystyka klasyfikacyjna):

$$\hat{k} = \arg \max_k S_k(\mathbf{x}) \quad (4)$$

¹Każda klasa na początku ma takie samo prawdopodobieństwo

Architektura PNN zapewnia brak problemu minimów lokalnych (sieć nie jest trenowana gradientowo) oraz asymptotyczną zbieżność do optymalnego klasyfikatora bayesowskiego wraz ze wzrostem liczebności zbioru uczącego [1].

2.2. Funkcje jądra

Funkcja jądra $K(x, x_i)$ określa, jak silnie wzorzec x_i wpływa na estymowaną gęstość w punkcie x [5]. Wybór funkcji jądra wpływa na kształt granicy decyzyjnej i odporność modelu na szum. Nie ma jednej uniwersalnej funkcji jądra, która byłaby najlepsza dla wszystkich problemów – dobór powinien być dostosowany do charakterystyki danych [1].

2.2.1. Gaussowska (Gaussian Kernel)

$$K(x, x_i) = \frac{1}{\sigma\sqrt{2\pi}} \cdot \exp\left(-\frac{\|x - x_i\|^2}{2\sigma^2}\right) \quad (5)$$

Najczęściej stosowana funkcja jądra [1], [6]. Gładka, różniczkowalna, symetrycznie zanika wraz z odległością. Wrażliwa na wartości odstające ze względu na szybki zanik gaussowski.

2.2.2. Laplasjańska (Laplacian Kernel)

$$K(x, x_i) = \frac{1}{2\sigma} \cdot \exp\left(-\frac{\|x - x_i\|}{\sigma}\right) \quad (6)$$

Zanika wolniej niż jądro gaussowskie – jest mniej czułe na wartości odstające i lepiej sprawdza się przy danych z rozkładami o grubych ogonach.

2.2.3. Cauchy'ego (Cauchy Kernel) [7]

$$K(x, x_i) = \frac{1}{\pi\sigma} \cdot \frac{1}{1 + \frac{\|x - x_i\|^2}{\sigma^2}} \quad (7)$$

Posiada bardzo grube ogony, przez co silnie uwzględnia odległe wzorce. Przydatne przy danych silnie zaszumionych lub heterogenicznych.

2.2.4. Odwrotna multikwadratowa (Inverse Multiquadric Kernel) [8]

$$K(x, x_i) = \frac{1}{\sigma} \cdot \frac{1}{\sqrt{\|x - x_i\|^2 + \sigma^2}} \quad (8)$$

Funkcja zawsze dodatnia o łagodnym zaniku. Stosowana jako alternatywa dla jądra gaussowskiego, gdy pożądana jest mniejsza czułość na dokładną wartość σ .

2.2.5. Epanechnikova (Epanechnikov Kernel) [9]

$$K(x, x_i) = \frac{3}{4\sigma} \cdot \max\left(0, 1 - \frac{\|x - x_i\|^2}{\sigma^2}\right) \quad (9)$$

Jądro jest gładkie wewnątrz obszaru wsparcia, ale na jego brzegu ma nieciągłość pochodnej. Z tego powodu dobrze sprawdza się w zadaniach, w których zależy nam na lokalnym uśrednianiu i ograniczeniu wpływu odległych obserwacji. W porównaniu z jądrem gaussowskim daje bardziej „lokalny” charakter estymacji i może lepiej tłumić słabo istotne, odległe wzorce.

Używamy funkcji max do zapewnienia, że jądro nie zwróci wartości ujemnej. Ponieważ wartości ujemne nie mają sensu w kontekście estymacji gęstości, funkcja max gwarantuje, że jądro będzie miało wartość zero dla obserwacji znajdujących się poza obszarem wsparcia (gdzie $\|x - x_i\|^2 > \sigma^2$).

2.2.6. Trójkątna (Triangular Kernel)

$$K(x, x_i) = \frac{1}{\sigma} \max\left(0, 1 - \frac{\|x - x_i\|}{\sigma}\right) \quad (10)$$

Jądro trójkątne jest funkcją liniową, która maleje wraz z odległością, osiągając wartość zero w odległości σ . Jest to jądro o ograniczonym zasięgu, które uwzględnia tylko obserwacje znajdujące się w promieniu σ od punktu estymacji. Dzięki temu jest szczególnie skuteczne w zadaniach, gdzie ważne jest uwzględnienie tylko lokalnych informacji i ograniczenie wpływu odległych obserwacji.

2.2.7. Jednostkowa (Uniform Kernel)

$$K(x, x_i) = \begin{cases} \frac{1}{2} & \text{kiedy } \|x - x_i\| \leq \sigma \\ 0 & \text{w przeciwnym wypadku} \end{cases} \quad (11)$$

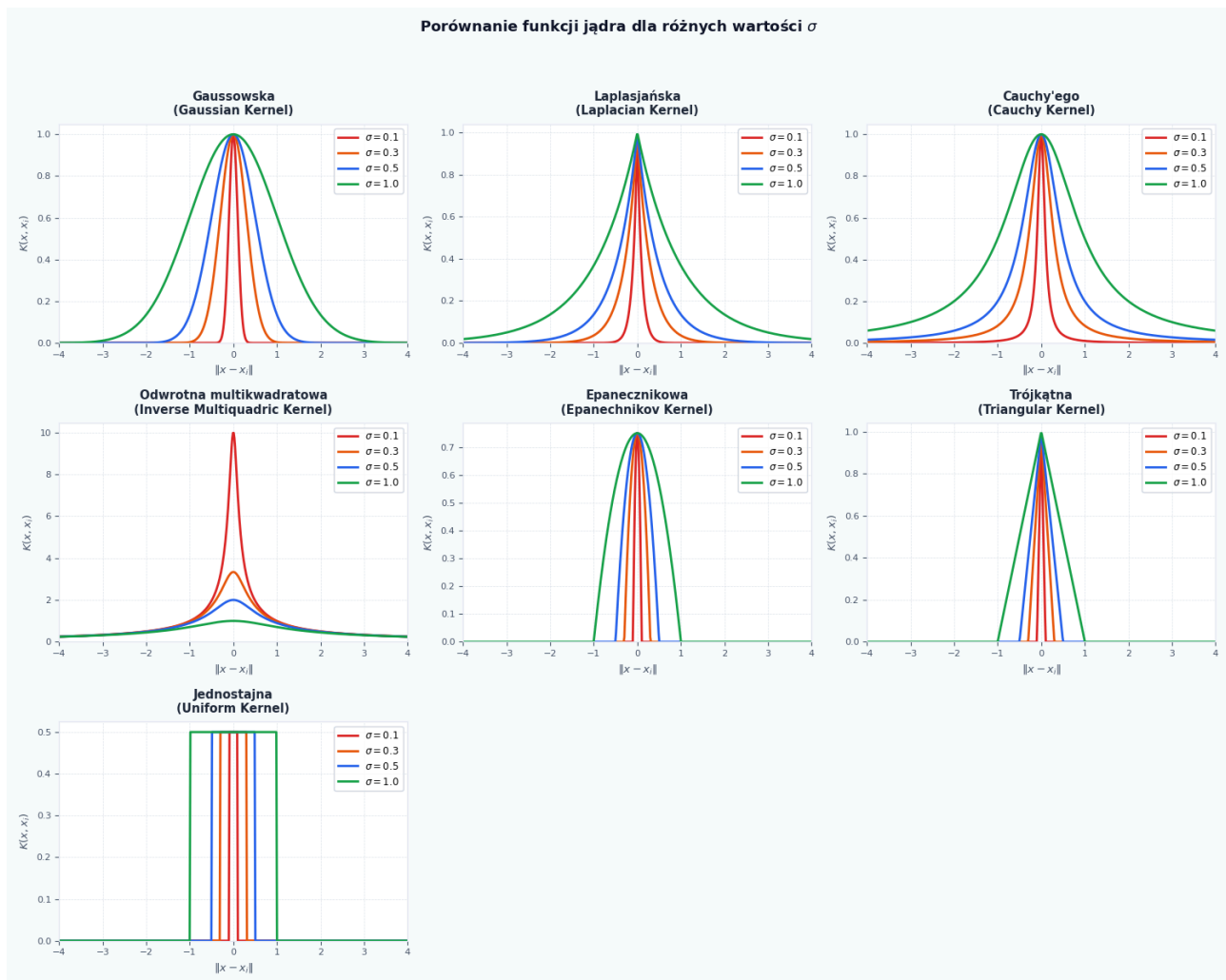
Jądro jednostkowe jest funkcją o stałej wartości wewnątrz obszaru wsparcia i zerowej wartości poza nim. Jest to najprostsze jądro, które uwzględnia tylko obserwacje znajdujące się w promieniu σ od punktu estymacji, przypisując im jednakową wagę. Ze względu na swoją prostotę, jądro jednostkowe może być mniej skuteczne w zadaniach, gdzie ważne jest uwzględnienie różnic w odległości między obserwacjami a punktem estymacji.

Porównanie kształtu różnych funkcji jądra przedstawiono w rysunku 2.

2.3. Parametr wygładzania σ

Parametr σ (parametr wygładzania, ang. **bandwidth, smoothing parameter**) jest kluczowym hiperparametrem PNN – kontroluje szerokość funkcji jądra i tym samym zasięg wpływu każdego wzorca treningowego na estymowaną gęstość [1]:

- **zbyt małe σ** – sieć dopasowuje się ściśle do zbioru uczącego (overfitting), granica decyzyjna jest nieregularna i wrażliwa na szum,
- **zbyt duże σ** – sieć nadmiernie wygładza rozkład (underfitting), tracąc lokalne struktury w danych.



Rysunek 2: Porównanie kształtu funkcji jądra dla kilku wartości σ

2.3.1. Dobór optymalnej wartości σ

Do wyznaczenia optymalnej wartości σ stosuje się zazwyczaj jedną z poniższych metod:

- **k -krotna walidacja krzyżowa** (ang. k -fold cross-validation) – wartość σ dobierana jest tak, aby minimalizować błąd klasyfikacji na zbiorze walidacyjnym [4],
- **reguła Silvermana** [5] – heurystyczny estymator oparty na rozrzucie danych po normalizacji:

$$\sigma^* = 1.06 * \hat{s} \cdot N^{-1/5} \quad (12)$$

gdzie $\hat{s}$ to skalarne odchylenie standardowe wszystkich elementów znormalizowanej macierzy cech (jak w implementacji: np. $\text{std}(x)$), a N to liczba próbek,

- **przeszukiwanie siatki** (ang. grid search) – systematyczne sprawdzenie zbioru kandydujących wartości σ z oceną na zbiorze testowym [4].

W praktyce zaleca się stosowanie walidacji krzyżowej jako metody najbardziej odpornej na specyfikę konkretnego zbioru danych.

2.4. Różnice między PNN a innymi sieciami neuronowymi

Sieć PNN różni się od tradycyjnych sieci neuronowych, takich jak perceptron wielowarstwowy (MLP) czy sieci konwolucyjne (CNN), przede wszystkim sposobem reprezentacji i przetwarzania danych:

- **Brak klasycznego uczenia gradientowego** – PNN nie wymaga iteracyjnego dostosowywania wag, dzięki czemu nie występuje problem minimów lokalnych, a budowa modelu przebiega szybciej,

- **Bezpośrednia reprezentacja zbioru uczącego** – każdy neuron wzorcowy odpowiada dokładnie jednemu przykładowi treningowemu, co pozwala od razu wykorzystać cały zbiór danych bez konieczności jego kompresji,
- **Estymacja gęstości zamiast funkcji decyzyjnej** – PNN opiera się na estymacji gęstości prawdopodobieństwa klas, natomiast MLP i CNN uczą się bezpośrednio funkcji decyzyjnej, co wpływa na kształt granic decyzyjnych oraz odporność modelu na szum.

PNN jest szczególnie skuteczna w zadaniach klasyfikacji, w których dostępna jest niewielka liczba danych, ponieważ tradycyjne sieci mogą mieć wtedy trudności z generalizacją. Z drugiej strony, ze względu na bezpośrednią reprezentację zbioru uczącego, PNN może być niepraktyczna w przypadku bardzo dużych zbiorów danych, głównie z uwagi na wysokie wymagania pamięciowe oraz dłuższy czas predykcji.

3. Analiza danych

Zbiór danych spambase [2] zawiera 4601 próbek wiadomości e-mail, z których około 39,4% to spam. Każda próbka jest reprezentowana przez 57 cech numerycznych, które opisują różne aspekty wiadomości, takie jak częstotliwość występowania określonych słów i znaków specjalnych, długość wiadomości. Celem jest klasyfikacja wiadomości jako spam lub nie-spam na podstawie tych cech.

Nr	Nazwa atrybutu	Opis	Zakres wartości
1–48	word_freq_SŁOWO	Procentowy udział danego słowa (make, free, you, credit, money, hp itp.) w całkowitej liczbie słów wiadomości. Obliczane jako $100 \cdot \frac{n_{\text{słowo}}}{n_{\text{słów}}}$. Zbiór zawiera 48 różnych słów charakterystycznych dla spamu i poczty służbowej.	[0, 100]
49–54	char_freq_ZNAK	Procentowy udział danego znaku (; ([! \$ #) w całkowitej liczbie znaków wiadomości. Obliczane jako $100 \cdot \frac{n_{\text{znak}}}{n_{\text{znaków}}}$. Znaki takie jak ! i \$ są silnymi wskaźnikami spamu.	[0, 100]
55	capital_run_length_average	Średnia długość nieprzerwanych sekwencji wielkich liter (wartość rzeczywista)	[1, 1102.5]
56	capital_run_length_longest	Długość najdłuższej nieprzerwanej sekwencji wielkich liter (wartość całkowita)	[1, 9989]
57	capital_run_length_total	Łączna liczba wielkich liter w całej wiadomości (wartość całkowita)	[1, 15841]
58	spam	Klasa wiadomości: 1 = wiadomość jest spamem, 0 = wiadomość nie jest spamem	{0, 1}

Tabela 1: Opis cech zbioru danych spambase [2]

Zbiór nie zawiera brakujących wartości, a cechy są numeryczne, co ułatwia ich bezpośrednie wykorzystanie w modelu PNN.

```
def load_data(path: str) -> tuple[np.ndarray, np.ndarray]:
    """
    Load data from a spambase-style .data file. The last column is the class label.
    :param path: Path to the .data file containing the data.
    :return: A tuple containing the features (X) and labels (y) as numpy arrays.
    """
    data = np.loadtxt(path, delimiter=",")
```



```
x = data[:, :-1] # All columns except the last one are features
y = data[:, -1] # The last column is the class label
return x, y
```

Listing 1: Metoda odpowiedzialna za wczytywanie danych z pliku .data

Dane pobierane są z pliku .data. Wszystkie cechy poza ostatnią są zwracane jako macierz z wymiarami $N \times p$, gdzie N to liczba próbek, a p to liczba cech (57). Ostatnia kolumna zawiera etykiety klas (0 lub 1) i jest zwracana jako wektor o długości N .

Dane przed przetwarzaniem są normalizowane, ponieważ cechy zbioru danych mają różne zakresy wartości (częstości słów i znaków: $[0, 100]$, natomiast cechy `capital_run_length_*` przyjmują wartości rzeczywiste do ok. 1.6×10^4). Jeżeli nie przeprowadzimy normalizacji, odległości euklidesowe obliczane przez PNN będą zdominowane przez cechy o większych zakresach, co może prowadzić do błędnych klasyfikacji. Normalizacja zapewnia, że wszystkie cechy mają równy wpływ na estymację gęstości i poprawia wydajność modelu.

```
def normalize_data(X: np.ndarray) -> np.ndarray:
    """
    Normalize the features in the dataset to have zero mean and unit variance.
    :param X: The input feature matrix to be normalized.
    :return: The normalized feature matrix.
    """
    mean = np.mean(X, axis=0)
    std = np.std(X, axis=0)
    std = np.where(std == 0, 1.0, std)
    return (X - mean) / std
```

Listing 2: Część skryptu odpowiedzialna za normalizację danych

Po znormalizowaniu danych, każda cecha ma wartość średnią równą 0 i odchylenie standardowe równe 1. Dzięki temu PNN może efektywnie obliczać odległości euklidesowe i estymować gęstość prawdopodobieństwa bez dominacji jednej cechy nad innymi.

4. Skrypt programu

```
class PNN:
    """
    Probabilistic Neural Network (PNN) implementation for classification tasks.
    Train with fit() method and predict with predict() method. Uses a kernel function to
    compute similarity between input and training patterns.
    """

    def __init__(self, kernel: Callable[[np.ndarray, np.ndarray, float], np.ndarray] =
PnnKernels.gaussian_kernel, sigma=0.1) -> None:
        """
        Initialize the PNN model with an optional kernel function and sigma parameter.
        :param kernel: The kernel function to use to compute similarity between input and
training patterns. Default is the Gaussian kernel.
        :param sigma: The sigma parameter for the kernel function (default is 0.1).
        """
        self.patterns : Optional[dict] = None
        self.sigma: float = sigma
        self.kernel = kernel
```

Listing 3: Klasa implementująca probabilistyczną sieć neuronową (PNN)

Parametr `self.kernel` odpowiada funkcji jądra używanej do estymacji gęstości; musi to być funkcja, która przyjmuje trzy argumenty: wektor wejściowy, wektor wzorca oraz wartość parametru wygładzania σ . Domyślnie jest to jądro Gaussa.

`self.sigma` to parametr wygładzania σ kontrolujący szerokość funkcji jądra z domyślną wartością 0.1.

Słownik `self.patterns` będzie przechowywał wzorce treningowe pogrupowane według klas.

```
def fit(self, x: np.ndarray, y: np.ndarray) -> None:
    """
    Fit (Train) the PNN model to the training data by storing the patterns for each
    class.
    :param x: Training data features
    :param y: Training data labels corresponding to the features in x.
    :return: None
    """
    self.patterns = {c: x[y == c] for c in np.unique(y)}
```

Listing 4: Metoda `fit()` odpowiedzialna za uczenie się sieci poprzez przypisanie wzorców do klas

Metoda `def fit()` jest odpowiedzialna za uczenie się sieci. Przypisuje ona każdej klasie jej wzorce, tworząc słownik, w którym kluczem jest etykieta klasy, a wartością jest macierz zawierająca wzorce treningowe należące do tej klasy.

```
def predict_single(self, x: np.ndarray) -> Optional[float]:
    """
    Predict the class label for a single input
    :param x: Input vector
    :return: Predicted class label
    """
    if self.patterns is None:
        raise ValueError("Model has not been fitted yet.")
    best_class : Optional[float] = None
    best_score = -np.inf
    for cls, patterns in self.patterns.items():
        score = np.sum(self.kernel(x, patterns, self.sigma))
        if score > best_score:
            best_score = score
            best_class = cls
    return best_class
```

Listing 5: Metoda `predict_single()` odpowiedzialna za przewidywanie klasy dla pojedynczej obserwacji

Metoda `def predict_single()` oblicza wynik S_k dla każdej klasy, sumując wartości funkcji jądra między wejściem a wzorcami danej klasy. Klasa z najwyższym S_k jest zwracana jako przewidywana etykieta.

```
def predict(self, x: np.ndarray) -> np.ndarray:
    """
    Predict the class labels for a set of input vectors.
    :param x: Array of input vectors to predict class labels for.
    :return: Array of predicted class labels corresponding to the input vectors.
    """
    return np.array([self.predict_single(xi) for xi in x])
```

Listing 6: Pomocnicza metoda `predict()`

Metoda `def predict()` jest pomocniczą funkcją, która umożliwia przewidywanie klas dla wielu obserwacji jednocześnie. Dla każdego wektora wejściowego w macierzy x wywołuje metodę `predict_single` i zwraca tablicę z przewidywanymi etykietami klas.

```
import numpy as np
```

```
class PnnKernels:
    @staticmethod
    def gaussian_kernel(x: np.ndarray, y: np.ndarray, sigma: float) -> np.ndarray:
        if sigma <= 0:
            raise ValueError("sigma must be > 0")
```

```

diff = x - y
sq_dist = np.sum(diff ** 2, axis=1)
norm_const = 1.0 / (sigma * np.sqrt(2.0 * np.pi))
return norm_const * np.exp(-sq_dist / (2 * sigma ** 2))

@staticmethod
def laplacian_kernel(x: np.ndarray, y: np.ndarray, sigma: float) -> np.ndarray:
    if sigma <= 0:
        raise ValueError("sigma must be > 0")

    ll_dist = np.sum(np.abs(x - y), axis=1)
    norm_const = 1.0 / (2.0 * sigma)
    return norm_const * np.exp(-ll_dist / sigma)

@staticmethod
def cauchy_kernel(x: np.ndarray, y: np.ndarray, sigma: float) -> np.ndarray:
    if sigma <= 0:
        raise ValueError("sigma must be > 0")

    sq_dist = np.sum((x - y) ** 2, axis=1)
    norm_const = 1.0 / (np.pi * sigma)
    return norm_const / (1.0 + sq_dist / (sigma ** 2))

@staticmethod
def inverse_multiquadric_kernel(x: np.ndarray, y: np.ndarray, c: float) -> np.ndarray:
    if c <= 0:
        raise ValueError("c must be > 0")

    sq_dist = np.sum((x - y) ** 2, axis=1)
    norm_const = 1.0 / c
    return norm_const / np.sqrt(sq_dist + c ** 2)

@staticmethod
def epanechnikov_kernel(x: np.ndarray, y: np.ndarray, h: float) -> np.ndarray:
    if h <= 0:
        raise ValueError("h must be > 0")

    sq_dist = np.sum((x - y) ** 2, axis=1)
    norm_const = 3.0 / (4.0 * h)
    return norm_const * np.maximum(0.0, 1.0 - sq_dist / (h ** 2))

@staticmethod
def triangular_kernel(x: np.ndarray, y: np.ndarray, h: float) -> np.ndarray:
    if h <= 0:
        raise ValueError("h must be > 0")

    l2_dist = np.linalg.norm(x - y, axis=1)
    norm_const = 1.0 / h
    return norm_const * np.maximum(0.0, 1.0 - l2_dist / h)

@staticmethod
def uniform_kernel(x: np.ndarray, y: np.ndarray, h: float) -> np.ndarray:
    if h <= 0:
        raise ValueError("h must be > 0")

    l2_dist = np.linalg.norm(x - y, axis=1)
    return np.where(l2_dist <= h, 0.5, 0.0)

```

Listing 7: Klasa implementująca funkcje jądra używane w probabilistycznej sieci neuronowej z
Rozdziału 2.2

Każda funkcja jądra jest zaimplementowana w osobnej metodzie statycznej klasy `PnnKernels`, co pozwala na łatwe dodawanie nowych jąder i eksperymentowanie z różnymi konfiguracjami. W każdej z tych funkcji korzystamy z `numpy` [10], aby przyspieszyć obliczenia i umożliwić efektywne przetwarzanie dużych zbiorów danych.

Pełny kod projektu dostępny jest w *dodatkach*

5. Eksperymenty

5.1. Badanie wpływu funkcji jądra oraz parametru wygładzania σ

W probabilistycznej sieci neuronowej kluczową rolę odgrywa parametr σ (Rozdział 2.3), który kontroluje szerokość funkcji jądra używanej do estymacji gęstości. Różne wartości σ mogą znacząco wpłynąć na dokładność klasyfikacji, dlatego ważne jest przeprowadzenie eksperymentów w celu znalezienia optymalnej wartości tego parametru. Porównamy również różne funkcje jądra, aby zobaczyć, jak wpływają one na wydajność modelu. (Rozdział 2.2).

5.1.1. Dobór optymalnej wartości σ i funkcji jądra za pomocą walidacji krzyżowej

Aby znaleźć najlepszą konfigurację jądra i wartości σ , przeprowadzimy eksperyment z walidacją krzyżową. Będziemy testować wartości σ w zakresie od 0.001 do 2 z krokiem 0.005, co pozwoli nam zobaczyć, jak dokładność zmienia się w szerokim zakresie wartości tego parametru. Dla każdej kombinacji funkcji jądra i wartości σ obliczymy dokładność klasyfikacji na zbiorze walidacyjnym i porównamy wyniki, aby znaleźć optymalną konfigurację.

```
def kFold_search(
    x: np.ndarray,
    y: np.ndarray,
    min_sigma: float = 0.001,
    max_sigma: float = 2,
    diff_sigma: float = 0.005,
    splits: int = 5,
    kernels=[PnnKernels.gaussian_kernel],
) -> dict[str, dict[float, np.ndarray]]:
    """
    Perform k-fold cross-validation to search for the best sigma and kernel configuration.
    :param x: The input feature matrix.
    :param y: The target labels.
    :param min_sigma: The minimum value for sigma.
    :param max_sigma: The maximum value for sigma.
    :param diff_sigma: The step size for sigma.
    :return: A dictionary containing the accuracies for each kernel and sigma combination.
    """
    accuracies: dict[
        str, dict[float, np.ndarray]
    ] = {} # {kernel_name: {sigma_value: (min_acc, max_acc, avg_acc)}}

    kf = KFold(n_splits=splits, shuffle=True, random_state=67)
    p = PNN()
    for kernel in kernels:
        kernel_name = kernel.__name__
        accuracies[kernel_name] = {}
        for sigma in np.arange(min_sigma, max_sigma + diff_sigma, diff_sigma):
            p.kernel = kernel
            p.sigma = sigma
            fold_accuracies = []
```

```

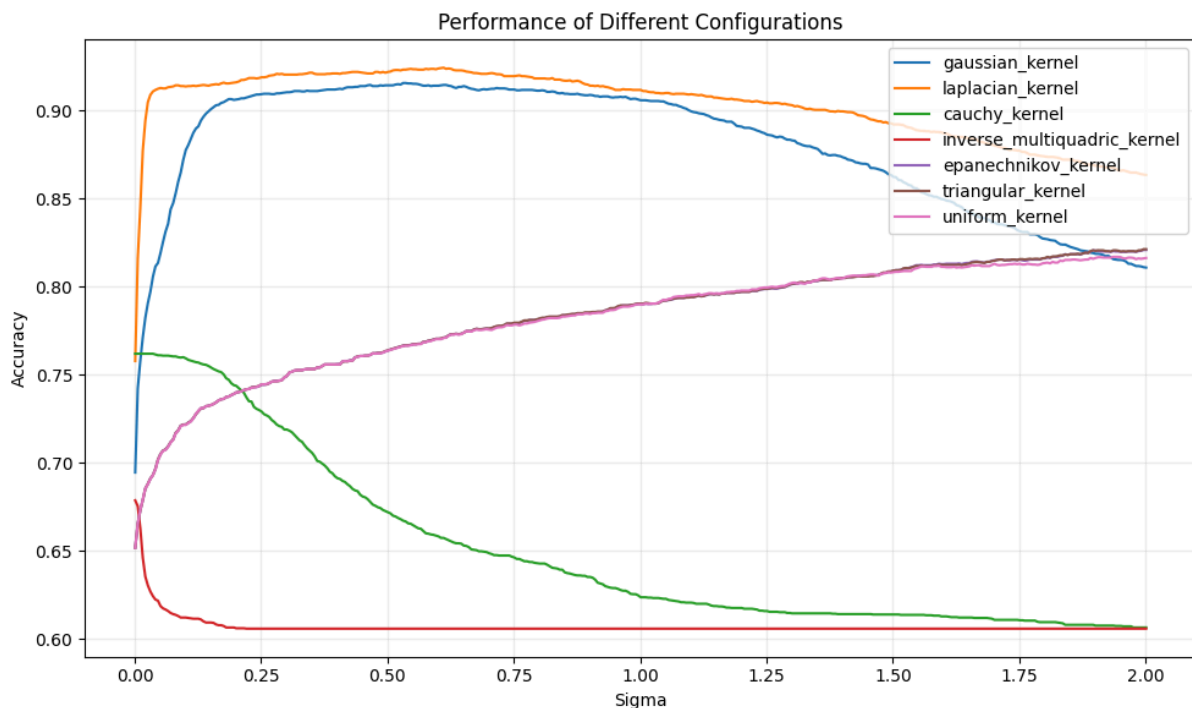
for train_index, test_index in kf.split(x):
    x_train, x_test = x[train_index], x[test_index]
    y_train, y_test = y[train_index], y[test_index]
    p.fit(x_train, y_train)
    y_pred = p.predict(x_test)
    fold_accuracy = np.mean(y_pred == y_test)
    fold_accuracies.append(fold_accuracy)
accuracies[kernel_name][sigma] = np.array(fold_accuracies)
print(
    f"Kernel: {kernel_name}, Sigma: {sigma:.3f}, Accuracy:
    {np.mean(accuracies[kernel_name][sigma]):.4f} (min: {np.min(accuracies[kernel_name]
    [sigma]):.4f}, max: {np.max(accuracies[kernel_name][sigma]):.4f})"
)
return accuracies

```

Listing 8: Część skryptu odpowiedzialna za poszukiwanie najlepszej konfiguracji jądra i σ z użyciem walidacji krzyżowej (zob. Rozdział 2.3.1)

k -krotna walidacja jest zaimplementowana za pomocą klasy `KFold` z biblioteki `sklearn.model_selection` [11], która dzieli dane na n losowych podzbiorów; w naszym przypadku $n = 5$. Dla każdej kombinacji funkcji jądra i wartości σ model jest trenowany na $n - 1$ podziorach, a następnie testowany na pozostałym podziorze. Proces ten jest powtarzany dla wszystkich możliwych podziałów danych, a dokładność jest obliczana jako średnia z wyników uzyskanych na wszystkich podziałach.

Badamy wartości σ w zakresie od 0.001 do 2 z krokiem 0.005, co pozwala nam zobaczyć, jak dokładność zmienia się w szerokim zakresie wartości tego parametru.



Rysunek 3: Wykres dokładności dla różnych konfiguracji jądra i wartości sigma.

Każda linia reprezentuje inną funkcję jądra, a oś x przedstawia wartości sigma. Każdy punkt na wykresie to średnia dokładność uzyskana z walidacji krzyżowej dla danej kombinacji jądra i sigma.

Najlepszą dokładność klasyfikacji osiągnięto dla **jądra laplasjańskiego przy $\sigma = 0.611$** , przy średniej dokładności **0.9244** (min: 0.9121, max: 0.9359). Ze względu na podobną charakterystykę jądra gaussowskiego, osiągnęło ono podobne wyniki, największa średnia dokładność dla jądra gaussowskiego wyniosła 0.9146 (min: 0.9088, max: 0.9250).

Jądra trójkątne, epanechnikowe i jednostkowe wykazały prawie taką samą dokładność na całym zakresie σ , co może być spowodowane ich podobną charakterystyką (zob. *Rysunek 2*). Najlepszy wynik z tych trzech funkcji uzyskało jądro trójkątne, które osiągnęło dokładność 0.8213 przy $\sigma \approx 2$. Co jest znacznie gorszym wynikiem niż jądra gaussowskie i laplasjańskie.

Jądra cauchy'ego i odwrotna multikwadratowa nie są odpowiednie dla tego zbioru danych, ich dokładność nie przekroczyła 0.8 dla żadnej wartości σ .

5.1.2. Wykorzystanie reguły Silvermana do oszacowania optymalnej wartości σ

Wykorzystaliśmy również regułę Silvermana (*Rozdział 2.3.1*) do oszacowania optymalnej wartości σ na podstawie odchylenia standardowego cech.

```
silverman = 1.06 * np.std(x) * (len(x) ** (-1 / 5))
```

Listing 9: Część skryptu odpowiedzialna za obliczenie wartości σ na podstawie reguły Silvermana

Obliczona wartość $\sigma^* = 0.1962$

Jądro	Dokładność
Gaussowska	0.9033
Laplasjańska	0.9141
Cauchy'ego	0.7511
Odwrotna multikwadratowa	0.6043
Epanechnikowa	0.7413
Trójkątna	0.7413
Jednostkowa	0.7413

Tabela 2: Dokładność klasyfikacji dla różnych funkcji jądra przy wartości σ oszacowanej za pomocą reguły Silvermana

Ponownie najlepszą dokładność osiągnęło jądro laplasjańskie. Dokładność jest mniejsza niż w przypadku optymalnego σ znalezionej za pomocą walidacji krzyżowej, ale nadal jest wysoka (0.9141).

$\Delta = 0.9244 - 0.9141 = 0.0103$, więc różnica jest niewielka, co potwierdza, że reguła Silvermana jest użytecznym narzędziem do oszacowania początkowej wartości σ , która następnie może być dostrojona za pomocą walidacji krzyżowej.

5.2. Badanie wpływu stałej normalizującej w funkcjach jądra

W prawie każdej definicji funkcji jądra uwzględnione są stałe normalizujące, które zapewniają, że jądro jest poprawnie znormalizowane jako estymator gęstości. Na przykład dla jądra gaussowskiego, stała normalizująca to $\frac{1}{\sigma\sqrt{2\pi}}$. Jednak w kontekście klasyfikacji, gdzie ostateczna decyzja opiera się na maksymalnej wartości estymowanej gęstości, te stałe nie powinny wpływać na wynik klasyfikacji, ponieważ są one takie same dla wszystkich klas i nie zmieniają relatywnych wartości estymowanych gęstości. Możemy przeprowadzić eksperyment, w którym porównamy dokładność klasyfikacji z uwzględnieniem stałych normalizujących i bez nich. Oczekujemy, że dokładność pozostanie taka sama, co potwierdzi, że te stałe nie mają wpływu na decyzję klasyfikacyjną.

Dane sprawdzimy dla jądra gaussowskiego i parametru $\sigma \in \{0.001, 0.101, 0.201, \dots, 2\}$

Nieznormalizowane jądro gaussowskie będzie miało postać:

$$K(\mathbf{x}, \mathbf{x}_i) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}_i\|^2}{2\sigma^2}\right) \quad (13)$$

```

# Check if you need normalization in kernels
kernels = [
    PnnKernels.gaussian_kernel,
    lambda x, y, sigma: np.exp(-np.sum((x - y) ** 2, axis=1) / (2 * sigma ** 2)), #
    Gaussian without normalization
]

accs = kFold_search(x, y, kerns=kernels, max_sigma=1, diff_sigma=0.1)
gauss_acc = accs[kernels[0].__name__]
no_norm_acc = accs[kernels[1].__name__]
# First ensure the same set of sigma keys, then compare arrays per key using allclose.
same_keys = set(gauss_acc.keys()) == set(no_norm_acc.keys())
if not same_keys:
    is_same = False
else:
    is_same = all(
        np.allclose(gauss_acc[sigma], no_norm_acc[sigma])
        for sigma in gauss_acc.keys()
    )
print(f"Gaussian kernel (with normalization) equals no-norm Gaussian? {is_same}")

```

Listing 10: Część skryptu odpowiedzialna za porównanie dokładności klasyfikacji z uwzględnieniem stałych normalizujących funkcje jądra i bez nich

Ponownie używamy metody `kFold_search` do porównania dokładności klasyfikacji dla jądra gaussowskiego z normalizacją i bez niej. Sprawdzamy, czy dokładności są takie same dla wszystkich wartości σ . Robimy to poprzez porównanie tablic dokładności dla obu wariantów jądra i sprawdzenie, czy są one bliskie (używając `np.allclose`). Jeśli dokładności są takie same dla wszystkich wartości σ , to potwierdza, że stałe normalizujące nie mają wpływu na dokładność klasyfikacji.

Po uruchomieniu skryptu otrzymujemy:

```
Gaussian kernel (with normalization) equals no-norm Gaussian? True
```

Co potwierdza, że stałe normalizujące nie mają wpływu na dokładność klasyfikacji, ponieważ oba warianty jądra dają taką samą dokładność.

6. Podsumowanie i wnioski

W ramach projektu przedstawiono teoretyczne podstawy probabilistycznej sieci neuronowej (PNN), jej architekturę oraz rolę funkcji jądra i parametru σ . Następnie przeprowadzono eksperyment porównujący kilka funkcji jądra dla różnych wartości σ .

Uzyskane wyniki potwierdzają, że skuteczność PNN silnie zależy od doboru hiperparametrów. Najlepszą dokładność klasyfikacji osiągnięto dla jądra laplasjańskiego przy odpowiednio dobranym σ , natomiast pozostałe jądra wykazywały większą wrażliwość lub niższą stabilność wyników.

Reguła Silvermana okazała się użytecznym narzędziem do oszacowania początkowej wartości σ , która następnie mogła być dostrojona za pomocą walidacji krzyżowej, co potwierdziło jej praktyczną wartość w kontekście PNN.

Sprawdziliśmy również, że stałe normalizujące w funkcjach jądra nie wpływają na dokładność klasyfikacji, co jest zgodne z teoretycznymi oczekiwaniami, ponieważ decyzja klasyfikacyjna opiera się na porównaniu wyników S_k , a te stałe są takie same dla wszystkich klas.

Probabilistyczne sieci neuronowe są skuteczną metodą klasyfikacji, podczas naszych eksperymentów osiągnęły wysoką dokładność porównywalną z innymi metodami klasyfikacji [2]. Mimo bardzo krótkiego czasu uczenia, PNN może być konkurencyjną alternatywą dla bardziej złożonych modeli, zwłaszcza w zadaniach z niewielką ilością danych.

Najważniejsze wnioski:

- dobór funkcji jądra ma istotny wpływ na jakość klasyfikacji,
- parametr σ powinien być strojony eksperymentalnie (np. walidacją krzyżową),
- PNN jest prostą i skuteczną metodą dla zadań klasyfikacji, szczególnie przy mniejszych zbiorach danych,
- ograniczeniem PNN pozostaje koszt pamięciowy i obliczeniowy przy dużej liczbie wzorców.

Bibliografia

- [1] D. F. Specht, „Probabilistic Neural Networks”, *Neural Networks*, t. 3, nr 1, s. 109–118, 1990.
- [2] M. Hopkins, E. Reeber, G. Forman, i J. Suermondt, „Spambase Data Set”. [Online]. Dostępne na: <https://archive.ics.uci.edu/dataset/94/spambase>
- [3] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York: Springer, 2006.
- [4] T. Hastie, R. Tibshirani, i J. Friedman, *The Elements of Statistical Learning*, 2. wyd. New York: Springer, 2009.
- [5] B. W. Silverman, *Density Estimation for Statistics and Data Analysis*. London: Chapman, Hall, 1986.
- [6] M. Kusy i R. Zajdel, „Probabilistic neural network training procedure based on Q(0)-learning algorithm in medical data classification”, *Applied Intelligence*, t. 41, s. 837–854, 2014, [Online]. Dostępne na: <https://api.semanticscholar.org/CorpusID:17015008>
- [7] F. Bach, „Are all kernels cursed?”. [Online]. Dostępne na: <https://francisbach.com/cursed-kernels/>
- [8] H. Hoffmann, „Common kernel functions ”. [Online]. Dostępne na: <https://www.heikohoffmann.de/htmlthesis/node39.html>
- [9] W. contributors, „Kernel Density Estimation”. [Online]. Dostępne na: [https://en.wikipedia.org/wiki/Kernel_\(statistics\)](https://en.wikipedia.org/wiki/Kernel_(statistics))
- [10] "NumPy developers", „NumPy Documentation”. [Online]. Dostępne na: <https://numpy.org/doc/2.4/reference/index.html#reference>
- [11] "Scikit-learn contributors", „sklearn.model_selection.KFold”. [Online]. Dostępne na: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.KFold.html

Dodatki

- [1] Repozytorium z kodem źródłowym projektu dostępne na https://github.com/NimVrod/PNN_S